

MARAN PMS — Prompt 50: Hospitalidad — Mejoras Operativas

Principio rector

NO reescribir el módulo. NO cambiar la estructura base existente. Solo mejoras incrementales sobre lo que ya funciona.

Archivos principales

- `client/src/pages/hospitality.tsx` — todo el módulo frontend
- `server/routes.ts` — endpoints `/api/hospitality/*`
- `server/storage.ts` — funciones de hospitalidad
- `shared/schema.ts` — tablas `guest_preferences` , `hospitality_alerts` , `stay_notes`

Lo que YA existe y NO tocar

- **Tablas:** `guest_preferences` , `hospitality_alerts` , `stay_notes`
 - `hospitality_alerts` **ya tiene:** `isAcknowledged` , `acknowledgedAt` , `acknowledgedBy`
 - `guest.segment` **ya existe con valores** `LEISURE` | `CORP` | `SPORT` | `CONGRESS` | `OTHER`
 - La integración check-in → generación automática de alertas ya funciona
 - El módulo tiene 4 tabs: Dashboard, Preferencias, Notas de Estadía, Historial
-

REGLA OPERATIVA CENTRAL (implementar primero)

Alcance de alertas y dashboard: solo huéspedes relevantes

Problema actual: el endpoint `/api/hospitality/dashboard` consulta **todos los huéspedes** del sistema para buscar preferencias críticas. Esto llena el dashboard con datos de huéspedes que no tienen ninguna estadía próxima, creando ruido operativo.

Regla: el dashboard y las alertas solo deben mostrar huéspedes con:

1. Check-in **hoy** o en los **próximos 7 días** (`confirmed` con `checkInDate` entre hoy y hoy+7)
2. Huéspedes actualmente **in-house** (`checked_in`)

Nadie más. Si un huésped frecuente tiene preferencias cargadas pero su próxima reserva es en 3 meses, no debe aparecer en el dashboard operativo.

Fix en `server/routes.ts` , endpoint `GET /api/hospitality/dashboard :`

```
app.get("/api/hospitality/dashboard", async (req, res) => {
  try {
    const today = new Date().toLocaleDateString("en-CA", {
      timeZone: "America/Argentina/Buenos_Aires"
    });
    const in7days = new Date();
    in7days.setDate(in7days.getDate() + 7);
    const limit = in7days.toLocaleDateString("en-CA", {
      timeZone: "America/Argentina/Buenos_Aires"
    });

    const allReservations = await storage.getReservations();

    // SOLO in-house + check-in en los próximos 7 días
    const relevantReservations = allReservations.filter(r =>
      r.status === "checked_in" ||
      (r.status === "confirmed" && r.checkInDate >= today && r.checkInDate <= lim
    );

    const guestsWithPrefs = [];
    for (const r of relevantReservations) {
      if (!r.guestId) continue;
      const guest = await storage.getGuest(r.guestId);
      const prefs = await storage.getActiveGuestPreferences(r.guestId);
      // Solo incluir si tiene preferencias activas
      if (guest && prefs.length > 0) {
        guestsWithPrefs.push({
          guest,
          reservation: r,
          preferences: prefs,
          isInHouse: r.status === "checked_in",
          checkInDate: r.checkInDate,
          hasCritical: prefs.some(p => p.priority === "critical"),
          hasHigh: prefs.some(p => p.priority === "high"),
          hasSpecialDate: prefs.some(p => p.category === "fecha_especial"),
```

```

        hasDiet: prefs.some(p => p.category === "alimentacion"),
    });
}
}

// Alertas pendientes solo de reservas relevantes
const relevantReservationIds = new Set(relevantReservations.map(r => r.id));
const allAlerts = await storage.getHospitalityAlerts();
const pendingAlerts = allAlerts.filter(a =>
    !a.isAcknowledged && relevantReservationIds.has(a.reservationId)
);

// Preferencias críticas: SOLO huéspedes con reserva relevante
const criticalPrefs = guestsWithPrefs
    .filter(g => g.hasCritical)
    .map(g => ({
        guest: g.guest,
        reservation: g.reservation,
        preferences: g.preferences.filter(p => p.priority === "critical"),
    }));

// Próximas fechas especiales (en los 7 días)
const upcomingSpecialDates = guestsWithPrefs
    .filter(g => g.hasSpecialDate)
    .map(g => ({
        guest: g.guest,
        reservation: g.reservation,
        specialDates: g.preferences.filter(p => p.category === "fecha_especial"),
    }));

res.json({
    inHouseGuests: guestsWithPrefs.filter(g => g.isInHouse),
    upcomingGuests: guestsWithPrefs.filter(g => !g.isInHouse),
    pendingAlerts,
    upcomingSpecialDates,
    criticalPreferences: criticalPrefs,
    stats: {
        totalInHouseWithPrefs: guestsWithPrefs.filter(g => g.isInHouse && g.prefs.length > 0).length,
        totalUpcomingWithPrefs: guestsWithPrefs.filter(g => !g.isInHouse).length,
        pendingAlertsCount: pendingAlerts.length,
        criticalCount: criticalPrefs.reduce((sum, g) => sum + g.preferences.length, 0),
    },
});
} catch (error) {
    res.status(500).json({ error: "Error fetching dashboard" });
}
});

```

Fix en el frontend (`DashboardTab`): actualizar para mostrar la nueva sección `upcomingGuests` con un separador visual claro entre "In-House ahora" y "Llegando en los próximos 7 días".

FASE 1 — CORRECCIONES CRÍTICAS

1. Estado de alertas: agregar `status` (`pending/in_progress/completed`)

Migración DB:

```
ALTER TABLE hospitality_alerts
  ADD COLUMN status TEXT NOT NULL DEFAULT 'pending';
-- valores: 'pending' | 'in_progress' | 'completed'
```

Schema (`shared/schema.ts`): agregar al schema existente de `hospitalityAlerts` :

```
status: text("status").notNull().default("pending"),
// 'pending' | 'in_progress' | 'completed'
```

Fix en `storage.ts` , función `acknowledgeHospitalityAlert` : cambiar estado a `in_progress` :

```
async acknowledgeHospitalityAlert(id: string, acknowledgedBy: string): Promise<Ho
  const [updated] = await db.update(hospitalityAlerts).set({
    isAcknowledged: true,
    acknowledgedAt: new Date(),
    acknowledgedBy,
    status: "in_progress", // ← cambiar de pending a in_progress
  }).where(eq(hospitalityAlerts.id, id)).returning();
  return updated;
}
```

Nuevo endpoint en `routes.ts` : marcar como completada:

```
app.patch("/api/hospitality/alerts/:alertId/complete", requireAuth, async (req, r
  try {
    const { completedBy, actionTaken } = req.body;
    const operador = completedBy || (req as any).user?.username || "Sistema";
    const [updated] = await db.update(hospitalityAlerts)
      .set({
        status: "completed",
```

```

        isAcknowledged: true,
        acknowledgedAt: new Date(),
        acknowledgedBy: operador,
        // Guardar la acción tomada en el campo notes si existe, o agregar campo
    })
    .where(eq(hospitalityAlerts.id, req.params.alertId))
    .returning();
    if (!updated) return res.status(404).json({ error: "Alert not found" });
    res.json(updated);
} catch (error) {
    res.status(500).json({ error: "Error completing alert" });
}
});

```

Fix en el frontend: el botón “Entendido” actual pasa a llamarse “En proceso” y cambia el estado a `in_progress` . Agregar un segundo botón “Completado” que usa el nuevo endpoint. Mostrar quién y cuándo confirmó:

```

// En AlertsList / cada alerta:
{alert.status === "pending" && (
    <Button size="sm" onClick={() => acknowledgeMutation.mutate(alert.id)}>
        En proceso
    </Button>
)}
{alert.status === "in_progress" && (
    <>
        <span className="text-xs text-muted-foreground">
            {alert.acknowledgedBy} - {formatDateTime(alert.acknowledgedAt)}
        </span>
        <Button size="sm" variant="outline" onClick={() => completeMutation.mutate(alert.id)}>
            <CheckCircle className="h-4 w-4 mr-1" /> Completado
        </Button>
    </>
)}
{alert.status === "completed" && (
    <span className="text-xs text-green-600 flex items-center gap-1">
        <CheckCircle className="h-3 w-3" />
        Completado por {alert.acknowledgedBy}
    </span>
)}

```

Fix en el frontend: el botón de acknowledge debe enviar el usuario real, no el `hardcodeado` "Recepción" . Usar el usuario de sesión:

```

// Obtener usuario de sesión

```

```
const { data: currentUser } = useQuery({ queryKey: ["/api/auth/me"] });

// En la mutation:
acknowledgedBy: currentUser?.username || currentUser?.firstName || "Recepción"
```

2. Historial de hospitalidad por estadía

La tabla `stay_notes` ya existe con campos: `reservationId`, `guestId`, `category`, `title`, `description`, `priority`, `isResolved`, `resolvedBy`, `resolvedAt`, `recordedBy`.

Lo que falta: cuando se genera una alerta al hacer check-in, crear también un registro en `stay_notes` vinculado a esa estadía.

Fix en el endpoint de check-in (`POST /api/reservations/:id/check-in` en `routes.ts`), dentro del bloque que genera alertas:

```
// Al generar cada alerta de hospitalidad, también crear stay_note:
for (const pref of preferences) {
  // ...código existente de crear alerta...

  // NUEVO: registrar en historial de la estadía
  await db.insert(stayNotes).values({
    id: randomUUID(),
    reservationId: req.params.id,
    guestId: reservation.guestId,
    category: pref.category as any,
    title: pref.title,
    description: pref.description || "",
    priority: pref.priority as any,
    visibleTo: pref.visibleTo || ["all"],
    isResolved: false,
    recordedBy: "sistema (check-in automático)",
    createdAt: new Date(),
  });
}
```

Fix en el tab "Historial" (`HistoryTab`): verificar que muestre el historial por huésped usando `stay_notes`. Si hoy muestra todas las notas sin agrupar por huésped/estadía, reorganizar para mostrar:

- Selector de huésped o búsqueda
- Por cada huésped: sus estadías con las notas de cada una

- Indicador de si cada nota se resolvió y quién la resolvió

FASE 2 — MEJORAS OPERATIVAS

3. Integración con planning: íconos de preferencias en reservas

En `client/src/pages/planning.tsx`, en el renderizado de cada celda/barra de reserva, agregar íconos pequeños cuando la reserva tiene preferencias activas.

El `planning` ya recibe datos de `reservationsMap` desde `/api/planning`. Extender ese objeto para incluir un resumen de preferencias:

Fix en `server/storage.ts`, función `getPlanningData`:

```
// En el loop de reservas, agregar preferencias resumidas:
for (const res of allReservations) {
  const guest = guestsMap.get(res.guestId);
  if (guest) {
    // Obtener preferencias activas
    const prefs = await this.getActiveGuestPreferences(res.guestId);

    reservationsMap[res.id] = {
      // ...campos existentes...
      prefSummary: prefs.length > 0 ? {
        hasCritical: prefs.some(p => p.priority === "critical"),
        hasHigh: prefs.some(p => p.priority === "high"),
        hasSpecialDate: prefs.some(p => p.category === "fecha_especial"),
        hasDiet: prefs.some(p => p.category === "alimentacion"),
        count: prefs.length,
      } : null,
    };
  }
}
```

Fix en planning frontend: en el `hover`/`tooltip` de la reserva (ya implementado en Prompt 44, punto 11), agregar `badges` de preferencias:

```
// Dentro del tooltip de reserva:
{res.prefSummary && (
  <div className="flex gap-1 mt-1">
    {res.prefSummary.hasCritical && (
      <span title="Preferencia crítica" className="text-red-500"></span>
    )}
  )}
```

```

    {res.prefSummary.hasSpecialDate && (
      <span title="Fecha especial" className="text-yellow-500">🎂</span>
    )}
    {res.prefSummary.hasDiet && (
      <span title="Restricción alimentaria" className="text-orange-500">🥗</span>
    )}
    {!res.prefSummary.hasCritical && res.prefSummary.hasHigh && (
      <span title="Preferencia alta" className="text-amber-500">⚠️</span>
    )}
  </div>
)}

```

Importante: NO agregar una nueva query al planning para esto. Los datos vienen en el mismo objeto `reservationsMap` ya cargado. Si agregar `getActiveGuestPreferences` en el loop de `getPlanningData` resulta en demasiadas queries (N+1), usar una sola query para traer todas las preferencias de una vez:

```

// En vez de N queries individuales:
const allPrefs = await db.select().from(guestPreferences)
  .where(and(
    eq(guestPreferences.isActive, true),
    inArray(guestPreferences.guestId, allReservations.map(r => r.guestId).filter(
    ));
const prefsByGuest = new Map<string, GuestPreference[]>();
for (const p of allPrefs) {
  if (!prefsByGuest.has(p.guestId)) prefsByGuest.set(p.guestId, []);
  prefsByGuest.get(p.guestId)!.push(p);
}
// Usar prefsByGuest.get(res.guestId) en el loop

```

4. Segmentación de huéspedes: badges de segmento

El campo `segment` ya existe en `guests` con valores `LEISURE` | `CORP` | `SPORT` | `CONGRESS` | `OTHER`.

Solo hay que agregar badges visuales en los lugares donde se muestra al huésped.

Mapeo de segmento a label y color:

```

const segmentConfig: Record<string, { label: string; className: string }> = {
  LEISURE: { label: "Leisure", className: "bg-blue-100 text-blue-700" },
  CORP: { label: "Corporativo", className: "bg-purple-100 text-purple-700" },
  SPORT: { label: "Deportivo", className: "bg-green-100 text-green-700" },

```



```
CONGRESS: { label: "Congreso",      className: "bg-amber-100 text-amber-700" },
OTHER:     { label: "Otro",          className: "bg-gray-100 text-gray-600" },
};
```

Agregar badge en:

1. **Dashboard de hospitalidad** — junto al nombre del huésped in-house
2. **Ficha de huésped** (`GuestsPage`) — en el card de detalle, junto al nombre
3. **Check-in** — en el detalle de la reserva antes de confirmar

No requiere ningún cambio de DB ni schema.

5. Fechas especiales: alertas el día del evento

En el endpoint `/api/hospitality/dashboard` ya incluido arriba, `upcomingSpecialDates` filtra las preferencias de tipo `fecha_especial` de los huéspedes con reserva relevante.

Agregar lógica de alerta automática cuando la fecha especial coincide con el día actual o la estadía actual:

En el endpoint de dashboard, dentro del loop de `guestsWithPrefs`:

```
// Si tiene fecha especial y hay coincidencia con hoy o mañana:
const today = new Date().toLocaleDateString("en-CA", { timeZone: "America/Argentina/Buenos_Aires" });
for (const pref of prefs.filter(p => p.category === "fecha_especial")) {
  // La descripción puede contener la fecha o el tipo (cumpleaños, aniversario, e
  // Por ahora: solo mostrar en dashboard destacado si el huésped está in-house
  // La generación automática de alertas se maneja desde check-in (ya existe)
}
```

En el **tab Dashboard**, agregar una sección visible “Fechas especiales hoy/esta semana” si `upcomingSpecialDates.length > 0`, con badge 🎂 y el nombre del huésped destacado.

FASE 3 — FICHA COMPLETA DEL HUÉSPED (no crítico)

6. Vista de historial en pestaña Historial

El tab “Historial” ya existe. Verificar que muestre:

- Lista de estadías pasadas del huésped seleccionado (desde `reservations` donde

```
guestId = id y status = checked_out )
```

- Para cada estadía: las notas de `stay_notes` vinculadas a esa `reservationId`
- Preferencias activas actuales del huésped

Si el tab hoy muestra algo distinto o está incompleto, ajustarlo para que la vista sea:

```
[Buscador de huésped]
```

```
→ Huésped seleccionado: Nombre — Segmento
```

```
→ Preferencias activas: [lista con badges]
```

```
→ Estadías anteriores:
```

```
- Check-in: 15/03/2025 | Check-out: 18/03/2025 | Hab. 305
```

```
- Notas: "Almohadas extras solicitadas — resuelto por Housekeeping"
```

```
- Check-in: 10/01/2025 | Check-out: 12/01/2025 | Hab. 210
```

```
- Sin notas registradas
```

No requiere nueva tabla — usa `reservations`, `stay_notes` y `guest_preferences` existentes.

NOTAS TÉCNICAS

- La regla de los 7 días es el cambio más importante — mejora radicalmente la utilidad del dashboard.
- El campo `status` en `hospitality_alerts` requiere migración con `DEFAULT 'pending'` para no afectar registros existentes.
- El badge de segmento NO requiere cambio de DB — `guests.segment` ya existe.
- La query N+1 en `getPlanningData` para preferencias es el principal riesgo de performance — usar la query batch con `inArray` como se indica.
- El endpoint de `complete` alerta usa `requireAuth` — verificar que los demás endpoints de hospitalidad también lo usen.
- NO agregar métricas de rendimiento (% alertas resueltas, tiempo promedio) — se descarta por no aportar valor operativo en esta etapa.